

Contemporary C++: *Learning Modern C++ in a Modern Way*



مدرس: سعید امراللہی بیوکی

Saeed Amrollahi Boyouki

www.Linkedin.com/in/saeed-amrollahi-boyouki

Agenda 10/30

Introduction to Functions: Part 1

- Functions
- Writing simple functions
- Inline functions References and further readings
- Q & A

120 min (incl. Q & A)



Functions

- Program organization: Data structures, Functions, User-defined types
- Function is one of fundamental tools for *abstraction* in C++.
- A **function** is a piece of program that has a name, and that another part of the program can *call*, or cause to run.

- function declaration:

- function name
- return type
- number and type of arguments

```
double sqrt(double); // just declaration
double sqrt(double x) { // decl. and def.
    // ... compute the square root of x and return it;
}
void exit(int exit_code); // declaration
```

- function definition:

- function declaration
- function body

Return type

Function name

Argument type
and name

```
long int sqr(int x)
{
    return x * x;
}
```

Function body

Function call

- Use `sqr` and `sqrt`.

```
double sr2 = sqrt(2); // call sqrt with argument double(2)
long s2 = sqr(2); // call sqr with argument 2
double sr3 = sqrt("three"); // error: sqrt() requires an argument of type double
array<int, 4> a = { 0, 1, 2, 3 };
long s4 = sqr(a); // error: sqr() requires an argument of type int
```

- The semantics of *argument passing* are identical to the semantics of *initialization*.
- The type of the definition and all declarations for a function must specify the same type.

Actual
argument

```
long int sqr(int x)
{
    return x * x;
}

int main()
{
    long X = sqr(10); // function call
}
```

Formal argument

Functions: some examples

```
/* compute a student's overall grade from  
   midterm and final exam grades and homework grade */  
double grade(double midterm, double final, double homework)  
{  
    return 0.2 * midterm + 0.4 * final + 0.4 * homework;  
}
```

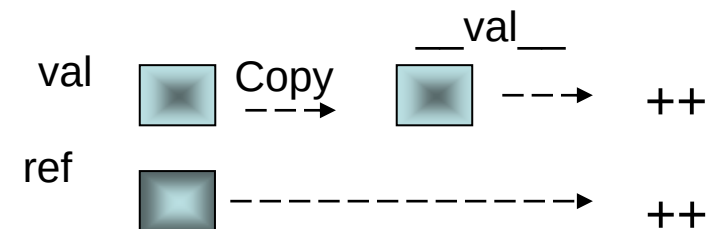
```
// return the maximum value of two integers  
int Max(int x, int y)  
{  
    return x > y ? x : y;  
}
```


Print Square Function
Prog.

Argument passing, pass by value, pass by reference

- Formal arguments vs. Actual arguments
- Initialization
- Type checking and type conversion
- Pass by value vs. Pass by reference

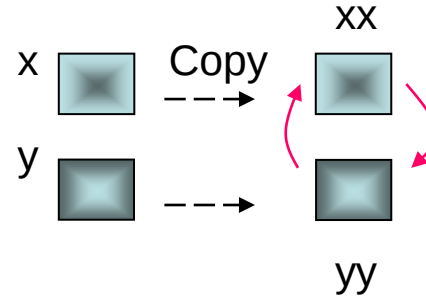
```
void f(int val, int& ref)
{
    val++;
    ref++;
}
void g()
{
    int i = 1;
    int j = 1;
    f(i,j); // i == 1, j == 2
}
```



Pass by value vs. Pass by reference: swap example

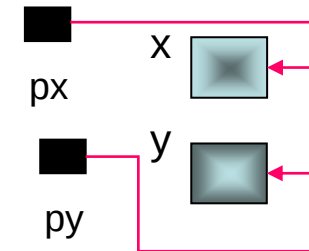
- Pass-by value

```
/* wrong */  
void swap(int x, int y)  
{  
    int tmp;  
    tmp = x;  
    x = y;  
    y = tmp;  
}
```



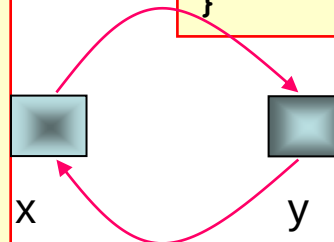
- Pass-by reference: C solution

```
/* right */  
void swap(int* px, int* py)  
{  
    int tmp;  
    tmp = *px;  
    *px = *py;  
    *py = tmp;  
}
```



- Pass-by reference: C++ solution

```
/* right */  
void swap(int& x, int& y)  
{  
    int tmp;  
    tmp = x;  
    x = y;  
    y = tmp;  
}
```



Return type, Value return and void

- A function should return a value unless it is a void function.
- void indicates absence of information.
- There are no objects of type void.

Return type

```
int f1() { } // error: no value returned
void f2() { } // ok
int f3() { return 1; } // ok
void f4() { return 1; } // error: return value in void function
int f5() { return; } // error: return value missing
void f6() { return; } // ok
```

Value return

- void has two main applications:
 - Function: void return type
 - Generic pointer: void*
- The semantics of *function value return* are identical to the semantics of *initialization*. A return statement is considered to initialize an *unnamed variable* of the returned type.
- Variable is an object that has a name.

Object vs. Variable

Value return, some points cont.

```
int* fp() // bad, don't do this
{
    int local = 1; // automatic variable
    /* ... */
    return &local; // return pointer to local
}
```

local destroyed/deallocated here

```
int& fp() // bad, don't do this
{
    int local = 1;
    /* ... */
    return local; // return reference to local
}
```

local destroyed/deallocated here

```
int fp() // good
{
    int local = 1;
    /* ... */
    return local; // OK: return a copy of local
}
```

Inline functions

- A function can be defined to be inline.
- `inline` is a function specifier. other specifiers are `virtual` and `explicit`.
- The inline specifier indicates to the implementation that inline substitution of the function body at the point of call is *to be preferred* to the usual function call mechanism.

```
// return the maximum value of two integers  
int Max(int x, int y)  
{  
    return x > y ? x : y;  
}
```

```
void f()  
{  
    int m = Max(3, 5); // function call  
}
```

Function call overhead



```
// return the maximum value of two integers  
inline int Max(int x, int y)  
{  
    return x > y ? x : y;  
}  
  
void f()  
{  
    int m = Max(3, 5); // expand inline  
}
```

Inline functions cont.

- The inline specifier is a *hint* to the compiler. Compilers are free to ignore inline directives and to make their own decisions about which functions to inline.
- An inline specifier does not affect the semantics of a function.
 - Inline function:
 - no function call overhead
 - larger program
 - faster program
 - Non-Inline function:
 - function call overhead
 - smaller program
 - slower program

Q Are inline functions guaranteed to make your performance better?

A No. Beware that overuse of inline functions can cause *code bloat*, which can in turn have a negative performance impact in paging environments.



R references and further readings

[KR78] Brian W. Kernighan & Dennis M. Ritchie. The C Programming Language, Prentice-Hall, Englewood Cliffs, NJ, 1978.

[Str00] Bjarne Stroustrup. The C++ Programming Language (Special Edition). Addison Wesley. Reading Mass. USA. February 2000.

[Str08] Bjarne Stroustrup. Programming: Principles and Practice in C++, 1st edition, Addison Wesley, December 2008.

[Str13] Bjarne Stroustrup. The C++ Programming Language. 4th edition. Addison Wesley. Reading Mass. USA. May 2013.

[Str22] Bjarne Stroustrup. A Tour of C++, 3rd edition, Addison Wesley, September 2022.

Thanks for your patience ...

A man who asks a question is a fool for minute,
The man who does not ask, is a fool for a life.
- Confucius

Learning to ask the right (often hard) questions is an essential part of learning to think as a programmer.

- Bjarne Stroustrup *programming Principles and Practice Using C++, page 4.*

There is no stupid question, but there is stupid answer.
- Howard Hinnant

